



Tecnológico de Monterrey

Marco de referencia
Desarrollo de aplicaciones avanzadas de ciencias computacionales

Alumnos:
Daniel Queijeiro Albo A01710441
Diego Isaac Fuentes Juvera
Diego Ricardo Alfaro Pito

Instituto Tecnológico y de Estudios Superiores de Monterrey,
Campus Querétaro

06/08/2026

Índice

1. Planteamiento del Problema	2
2. Antecedentes y Estado del Arte	2
2.1 Enfoques de Detección de Plagio en Código Fuente	2
2.2 Herramienta Fundacional: Dolos	3
2.3 Algoritmo Winnowing	4
3. Justificación	4
4. Objetivos	5
4.1 Objetivo General	5
4.2 Objetivos Específicos	5
5. Variables de Investigación	6
5.1 Variables Independientes (parámetros del modelo)	6
5.2 Variables Dependientes (métricas de desempeño)	7
5.3 Variable de Control	7
6. Metodología de Investigación	7
Etapa 1 — Revisión de literatura y diseño del marco de referencia	7
Etapa 2 — Construcción del pipeline de preprocesamiento	8
Etapa 3 — Implementación del motor de similitud	8
Etapa 4 — Desarrollo del modelo de aprendizaje automático	8
Etapa 5 — Evaluación experimental	8
Etapa 6 — Documentación y presentación	8
7. Hipótesis	8
8. Parámetros y Métricas de Evaluación	9
8.1 Dataset de Evaluación	9
8.2 Protocolo de Evaluación	10
8.3 Métricas Principales	10
8.4 Línea Base de Comparación	10
9. Alcance y Limitaciones	11
Alcance	11
Limitaciones	11
10. Referencias	12

1. Planteamiento del Problema

El plagio en código fuente es una práctica documentada y extendida en entornos educativos y de desarrollo de software. Estudios previos han demostrado que los sistemas de software contienen fragmentos de código plagiado en proporciones que pueden oscilar entre el 5% y el 20%. Esta problemática no solo representa una violación ética y legal, sino que también compromete la integridad académica y dificulta el mantenimiento y evolución de los sistemas de software.

A pesar de la existencia de herramientas de detección como MOSS (*Measure Of Software Similarity*) y JPlag, su adopción en ambientes educativos es limitada debido a barreras de accesibilidad, dependencia del lenguaje de programación, y falta de visualizaciones interpretables para los docentes. Adicionalmente, la mayoría de los enfoques tradicionales se centran en análisis textual y estructural superficial, lo que los hace vulnerables a transformaciones simples del código (renombrado de variables, reordenamiento de bloques, etc.).

Pregunta de investigación central:

¿Es posible diseñar e implementar un modelo computacional que, integrando técnicas de análisis de compiladores (AST), métodos cuantitativos y aprendizaje automático, detecte efectivamente el plagio en archivos de código fuente escritos en Python, superando o igualando el desempeño reportado por herramientas de referencia como Dolos y JPlag?

2. Antecedentes y Estado del Arte

2.1 Enfoques de Detección de Plagio en Código Fuente

La detección de plagio en código fuente ha evolucionado desde comparaciones léxicas simples hacia métodos semánticos y estructurales más robustos. Los enfoques principales se clasifican en:

- **Comparación textual y léxica:** Normalizan el código eliminando espacios y comentarios, y aplican métricas de similitud de cadenas. Son los más rápidos pero los más vulnerables a la ofuscación.
- **Tokenización:** Convierten el código en secuencias de tokens abstractos, eliminando información irrelevante como nombres de variables. MOSS utiliza esta estrategia con el algoritmo Winnowing.
- **Análisis de Árbol de Sintaxis Abstracta (AST):** Transforman el código en una representación jerárquica de su estructura sintáctica, independiente de nombres y formato. Es más resistente a obfuscariones que el análisis textual.
- **Basados en grafos (PDG/CFG):** Emplean grafos de dependencia de programa o flujo de control; ofrecen mayor precisión semántica pero mayor costo computacional.
- **Aprendizaje automático:** Modelos entrenados para clasificar pares de código como plagio / no plagio a partir de características extraídas por los enfoques anteriores.

2.2 Herramienta Fundacional: Dolos

Dolos es una herramienta de detección de plagio en código fuente publicada en el *Journal of Computer Assisted Learning* que introduce un enfoque agnóstico al lenguaje de programación. Sus características clave son:

Pipeline de procesamiento:

1. **Parsing con modelos genéricos (Tree-sitter):** Dolos utiliza la librería Tree-sitter para generar Árboles de Sintaxis Concreta (CST) de manera independiente del lenguaje. Esto le permite soportar una amplia gama de lenguajes sin requerir parsers específicos.
2. **Tokenización y enmascaramiento:** Los nodos del CST son serializados en una secuencia de tokens. Los identificadores (nombres de variables, funciones) son enmascarados para que el análisis sea insensible a renombramientos.
3. **Generación de huellas digitales (Fingerprinting) con Winnowing:** Se calculan *k-gramas* (subcadenas de *k* tokens consecutivos) que son convertidos en hashes. El algoritmo Winnowing muestrea estos hashes de manera eficiente, seleccionando el de menor valor en cada ventana deslizante de tamaño *w*. El

parámetro por defecto es $k = 23$. Este proceso produce una "huella digital" compacta de cada archivo.

4. **Cómputo de similitud por pares:** La similitud entre dos archivos se calcula como la fracción de huellas compartidas sobre el total de huellas de cada archivo. Este cálculo es inmune al reordenamiento de bloques de código gracias a la naturaleza de las subcadenas compartidas.

Benchmarking: Dolos fue evaluado sobre el conjunto de datos SOCO, logrando un F1-score de 0.865 en Java, superando a JPlag y MOSS en las métricas de precisión y recall.

Visualizaciones: La herramienta provee un tablero interactivo que permite a los docentes explorar pares sospechosos, visualizar fragmentos compartidos lado a lado y analizar clusters de similitud.

2.3 Algoritmo Winnowing

El algoritmo Winnowing es fundamental para la eficiencia de Dolos. Dado un flujo de tokens:

- Se calculan hashes de cada *k-grama* (subsecuencia de k tokens).
- Se aplica una ventana deslizante de tamaño w sobre los hashes.
- De cada ventana se retiene únicamente el hash mínimo.
- El conjunto de hashes retenidos constituye la huella digital del documento.

Sus propiedades garantizan que cualquier subcadena de longitud $\geq k$ compartida entre dos documentos será detectada, manteniendo un balance óptimo entre tasa de detección y compresión (se retiene aproximadamente 1 de cada w hashes).

3. Justificación

El presente proyecto se justifica en tres dimensiones:

Académica: La detección automática de plagio en código es esencial para preservar la integridad en cursos de programación, especialmente en ambientes de aprendizaje en línea donde la supervisión directa es limitada.

Técnica: La implementación de un sistema propio, basado en los fundamentos de Dolos pero especializado para Python, permite explorar el diseño de compiladores (análisis léxico y sintáctico), métodos cuantitativos (hashing, similitud, estadística) y aprendizaje automático (clasificación de pares) en un problema real e integrado.

Formativa: El reto integra de manera coherente los cuatro módulos del curso: metodología de investigación (diseño experimental y evaluación), inteligencia artificial (modelos de clasificación), compiladores (AST y tokenización) y métodos cuantitativos (métricas y algoritmos de hashing).

4. Objetivos

4.1 Objetivo General

Desarrollar un modelo computacional capaz de detectar plagio en archivos de código fuente escritos en Python, integrando análisis de Árboles de Sintaxis Abstracta, el algoritmo Winnowing para generación de huellas digitales, y técnicas de aprendizaje automático para la clasificación de pares sospechosos; evaluando su desempeño mediante métricas estándar sobre un conjunto de datos etiquetado.

4.2 Objetivos Específicos

1. **Construir el pipeline de preprocesamiento:** Implementar un tokenizador basado en el módulo “ast” de Python (o Tree-sitter) que genere secuencias de tokens normalizados a partir de archivos “.py”, enmascarando identificadores.
2. **Implementar el algoritmo Winnowing:** Desarrollar la generación de *k-gramas*, el cómputo de hashes rodantes y la selección de huellas digitales para cada archivo de código fuente.

3. **Calcular matrices de similitud:** Implementar el cómputo de similitud por pares sobre conjuntos de archivos, representando los resultados como matriz de adyacencia y grafo de similitud.
 4. **Integrar un modelo de aprendizaje automático:** Entrenar y evaluar un clasificador (p. ej., Random Forest, SVM o red neuronal) que, a partir de las métricas de similitud extraídas, clasifique pares de archivos como plagio o no plagio.
 5. **Evaluar el modelo propuesto:** Medir el desempeño del sistema mediante Precisión, Recall, F1-score y AUC-ROC sobre un dataset etiquetado, comparando los resultados con el umbral de referencia reportado por Dolos (F1 = 0.865 en Java).
 6. **Documentar el proceso de investigación:** Elaborar el Informe de Desarrollo y Evaluación del Modelo Computacional con los resultados experimentales y su análisis estadístico.
-

5. Variables de Investigación

5.1 Variables Independientes (parámetros del modelo)

Variable	Descripción	Valores a explorar
k (tamaño de k-grama)	Longitud de la subcadena de tokens para el hashing	5, 10, 15, 23 (default Dolos), 30
w (tamaño de ventana Winnowing)	Tamaño de la ventana deslizante para selección de huellas	4, 8, 16
Nivel de enmascaramiento	Qué elementos del AST se normalizan (solo variables, también literales, etc.)	Bajo, Medio, Alto
Algoritmo de clasificación ML	Modelo utilizado para la decisión final de plagio	Random Forest, SVM, Regresión Logística

5.2 Variables Dependientes (métricas de desempeño)

Variable	Descripción	Fórmula
Precisión	Fracción de pares detectados como plagio que realmente lo son	$TP / (TP + FP)$
Recall (Sensibilidad)	Fracción de pares de plagio real que fueron detectados	$TP / (TP + FN)$
F1-score	Media armónica de Precisión y Recall	$2 \cdot (P \cdot R) / (P + R)$
AUC-ROC	Área bajo la curva ROC; evalúa el poder discriminativo del modelo	—
Tiempo de procesamiento	Tiempo de análisis por par de archivos	segundos/par

5.3 Variable de Control

- **Lenguaje de programación analizado:** Python exclusivamente.
 - **Conjunto de datos:** Dataset etiquetado fijo (SOCO o dataset generado ad hoc con transformaciones de código documentadas).
-

6. Metodología de Investigación

El proyecto sigue un **enfoque cuantitativo experimental**, con diseño de tipo comparativo-evaluativo. Las etapas son:

Etapas 1 — Revisión de literatura y diseño del marco de referencia

Análisis del artículo fundacional de Dolos y de trabajos relacionados (JPlag, MOSS, técnicas de AST). Definición de preguntas de investigación, hipótesis, variables y métricas de evaluación.

Etapas 2 — Construcción del pipeline de preprocesamiento

Implementación del parser de Python con el módulo ast o Tree-sitter. Diseño del esquema de tokenización y enmascaramiento. Validación del tokenizador con casos de prueba simples.

Etapas 3 — Implementación del motor de similitud

Implementación del algoritmo WInnowing. Construcción de la matriz de similitud por pares. Visualización de la distribución de similitudes.

Etapas 4 — Desarrollo del modelo de aprendizaje automático

Definición y extracción del vector de características (similitud WInnowing, cobertura de fragmentos compartidos, etc.). Entrenamiento y validación cruzada de los clasificadores. Optimización de hiperparámetros.

Etapas 5 — Evaluación experimental

Prueba del sistema completo sobre el dataset etiquetado. Cómputo de todas las métricas de desempeño. Análisis estadístico y comparación con la línea base de Dolos.

Etapas 6 — Documentación y presentación

Redacción del Informe de Desarrollo y Evaluación del Modelo Computacional. Preparación del prototipo de software y presentación final.

7. Hipótesis

H₁: Un modelo computacional que combina análisis de AST de Python, el algoritmo WInnowing con parámetros $k \in \{15, 23\}$ y un clasificador de aprendizaje automático alcanzará un F1-score ≥ 0.80 en la detección de plagio sobre el conjunto de datos de evaluación.

H_0 : Los parámetros del modelo (k , w , algoritmo ML) no tienen efecto significativo sobre el F1-score del sistema de detección de plagio ($F1 < 0.60$).

8. Parámetros y Métricas de Evaluación

8.1 Dataset de Evaluación

Se utilizará el conjunto de datos **Code Similarity Dataset – Python Variants**, disponible públicamente en Kaggle

(<https://www.kaggle.com/datasets/hemajitpatel/code-similarity-dataset-python-variants>).

Este dataset contiene múltiples problemas de programación en Python, cada uno con **20 variantes de solución por tarea**, lo que lo hace especialmente adecuado para el objetivo de este proyecto.

Estructura del dataset:

El dataset está organizado por problema, donde cada problema tiene N variantes que representan distintas formas de escribir soluciones funcionalmente equivalentes. Estas variantes simulan las técnicas de ofuscación más comunes en escenarios de plagio académico, incluyendo:

- Renombrado de variables e identificadores.
- Reordenamiento de bloques de código independientes.
- Variaciones en estructuras de control equivalentes (p. ej., $\text{for} \leftrightarrow \text{while}$, comprensiones de lista vs. bucles).
- Inserción de comentarios, líneas vacías o código muerto.

Construcción de pares etiquetados:

A partir de la estructura multi-problema del dataset se generarán dos categorías de pares:

Tipo de par	Criterio	Etiqueta
Par positivo (plagio)	Dos variantes del mismo problema	1

Par negativo (no plagio)	Dos soluciones de distintos problemas	0
---------------------------------	--	---

Dado que el dataset contiene 20 variantes por problema, cada problema genera $C(20,2) = 190$ **pares positivos**. Los pares negativos se muestrean aleatoriamente entre problemas distintos, balanceando las clases para el entrenamiento del clasificador ML.

8.2 Protocolo de Evaluación

- **Validación cruzada:** K-fold con $K = 5$ para la etapa de entrenamiento del clasificador ML.
- **Umbral de decisión:** Se determinará el umbral óptimo de similitud que maximiza el F1-score sobre el conjunto de validación.
- **Conjunto de prueba:** 20% del dataset, sin exposición durante el entrenamiento.

8.3 Métricas Principales

Métrica	Descripción	Meta del proyecto
Precisión	Exactitud de las alertas generadas	≥ 0.80
Recall	Cobertura de casos reales de plagio	≥ 0.75
F1-score	Balance entre Precisión y Recall	≥ 0.80
AUC-ROC	Discriminación global del modelo	≥ 0.85

8.4 Línea Base de Comparación

La línea base de referencia es el desempeño reportado por Dolos en el artículo fundacional: **F1 = 0.865** en el benchmark SOCO con archivos Java. Dado que el dataset de Kaggle está compuesto íntegramente por código Python con variantes controladas, se espera que un modelo bien ajustado pueda alcanzar o superar ese umbral. Se aceptará

como exitoso un desempeño $F1 \geq 0.80$ sobre el conjunto de prueba del dataset de Python Variants.

Adicionalmente, como validación cruzada de herramientas, se ejecutará Dolos directamente sobre un subconjunto del dataset para obtener su F1 empírico en Python, y se usará ese valor como segunda línea base más realista y comparable con el sistema propuesto.

9. Alcance y Limitaciones

Alcance

- El sistema analiza exclusivamente archivos de código fuente en **Python** (.py).
- La detección se realiza a nivel de **par de archivos** (similitud binaria plagio/no plagio), no a nivel de fragmento individual.
- El prototipo es una herramienta de apoyo académico; sus resultados no constituyen prueba definitiva de plagio.

Limitaciones

- El análisis semántico profundo (equivalencia lógica de algoritmos distintos con estructuras diferentes) está fuera del alcance de este proyecto.
 - El dataset de Kaggle contiene variantes sintéticas y controladas; la generalización a código real de producción o a estilos de plagio no contemplados en el dataset requiere evaluación adicional.
 - El sistema no contempla la detección de plagio proveniente de fuentes externas (Internet, repositorios públicos), únicamente entre archivos del conjunto analizado.
 - El balance artificial de pares positivos/negativos puede afectar la generalización si la distribución real de plagio en un ambiente académico difiere significativamente.
-

10. Referencias

Maertens, R., Van Petegem, C., Strijbol, N., Baeyens, T., Jacobs, A. C., Dawyndt, P., & Mesuere, B. (2022). Dolos: Language-agnostic plagiarism detection in source code.

Journal of Computer Assisted Learning, 38(4), 1046–1061.

<https://doi.org/10.1111/jcal.12662>

Maertens, R., Van Neyghem, M., Geldhof, M., Van Petegem, C., Strijbol, N., Dawyndt, P., & Mesuere, B. (2024). Discovering and exploring cases of educational source code plagiarism with Dolos. *SoftwareX*, 26, 101755. <https://doi.org/10.1016/j.softx.2024.101755>

Patel, H. (2025). *Code Similarity Dataset – Python Variants* [Dataset]. Kaggle.

<https://www.kaggle.com/datasets/hemajitpatel/code-similarity-dataset-python-variants>

Prechelt, L., Malpohl, G., & Philippsen, M. (2002). Finding plagiarisms among a set of programs with JPlag. *Journal of Universal Computer Science*, 8(11), 1016–1038.

<https://doi.org/10.3217/jucs-008-11-1016>

Schleimer, S., Wilkerson, D. S., & Aiken, A. (2003). Winnowing: Local algorithms for document fingerprinting. *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*, 76–85. <https://doi.org/10.1145/872757.872770>

Wise, M. J. (1996). YAP3: Improved detection of similarities in computer program and other texts. *Proceedings of the 27th SIGCSE Technical Symposium on Computer Science Education*, 130–134. <https://doi.org/10.1145/236452.236525>